

— Web apps

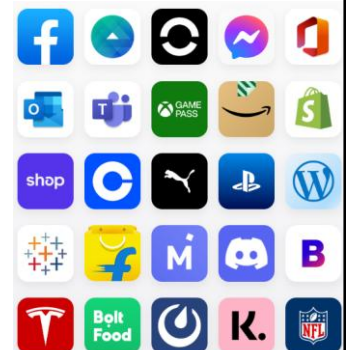
Chapter 9: React Native – introduction

L. Espeel



— React Native ⁽¹⁾

- Ontwikkeld door Meta Platforms (voorheen Facebook).
 - Sinds 2025 onderdeel van de React Foundation (Linux Foundation)
- Framework voor mobiele applicaties met JavaScript / TypeScript.
- Gebruikt React-concepten om native componenten aan te sturen.
- Ondersteunt Android, iOS, macOS, Windows en Web.
- Apps werken (en voelen aan) als echte native apps.
- Wordt tegenwoordig gebruikt in heel wat (commerciële) apps:



— React Native (2)

Voordelen:

- Eén codebase → meerdere platformen.
- Snelle ontwikkeling met Hot Reloading.
- Native performance (geen webview).
- Grote community & veel bibliotheken.
- Herbruikbare logica vanuit React.

Beperkingen:

- Niet alle native features zijn standaard beschikbaar (soms moet je zelf native code schrijven).
- Minder geschikt voor zeer grafisch intensieve apps (zoals games).

3

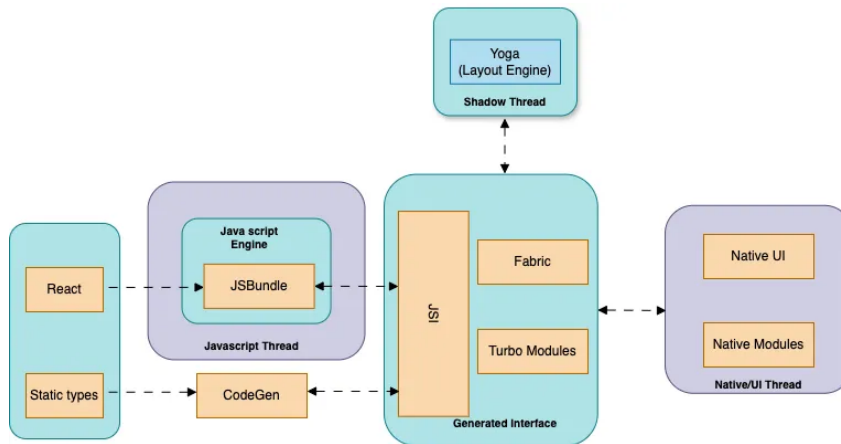
— Architectuur (1)

Van architectuur zijn er zaken anders in React Native t.o.v. React:

React (web)	React Native (mobile)	Toelichting
HTML + CSS	Native Views & Styles (via Fabric)	Fabric rendert direct native UI-elementen met native styling.
DOM	JSI (JavaScript Interface)	JSI is een snelle C++ interface die directe synchronisatie tussen JS en native mogelijk maakt.
React Router	React Navigation	Wordt nog steeds gebruikt voor navigatie, maar werkt bovenop de Fabric rendering engine (Fabric).
Browser	Android/iOS Emulator of fysiek device	De runtime-omgeving is een mobiele engine (Hermes, JSC of V8) die via JSI met native code praat.
	TurboModules	Om native modules dynamisch te laden via JSI.
	CodeGen	Genereert type-safe bindings tussen JS en native code.
	Yoga (Layout Engine)	Wordt gebruikt voor layout-berekeningen, is nu geïntegreerd met Fabric.

4

— Architectuur (2)



5

— Ontwikkeltools (1)

We kiezen Expo als ontwikkelplatform voor React Native:

- Maakt het eenvoudig om te starten, biedt standaardmodules en testmogelijkheden en laat apps draaien in de browser of Expo Go.
 - Expo Go: mobiele app (beschikbaar voor iOS en Android) waarmee je React Native-apps kunt bekijken, testen en ontwikkelen zonder dat je ze hoeft te bouwen of te installeren via Xcode of Android Studio.
- Bevat EAS (Expo Application Services) om builds te maken.
 - EAS is een verzameling cloud-diensten die door Expo wordt aangeboden om het bouwen, testen en publiceren van React Native-apps makkelijker en sneller te maken.

6

— Ontwikkeltools (2)

Volg de stappen op:

<https://docs.expo.dev/get-started/set-up-your-environment>

We hebben voor het voorbeeld hier verder gekozen voor:

- Android Emulator
- Development build
- Uitvinken optie *Build with Expo Application Services (EAS)* → we gaan een lokale build maken

7

— Ontwikkeltools (3)

- Kies als besturingssysteem bv. Windows voor de verdere instructies.
Zorg dat Openjdk17 geïnstalleerd wordt, dit kan via het choco-commando ofwel via <https://aka.ms/download-jdk/microsoft-jdk-17.0.16-windows-x64.msi>
- Je zal ook Android Studio nodig hebben met Android Emulator:
<https://redirector.gvt1.com/edgedl/android/studio/install/2025.1.3.7/android-studio-2025.1.3.7-windows.exe>
- Volg de stappen hoe je de SDK en emulator instelt in Android Studio:
<https://docs.expo.dev/get-started/set-up-your-environment/?platform=android&device=simulated&mode=development-build&buildEnv=local#set-up-android-studio>

8

— Aanmaken project ⁽¹⁾

Commando's :

- `npx create-expo-app@latest <name> --template <template type>`

in het verder voorbeeld in dit hoofdstuk hebben we dit commando gebruikt:

`npx create-expo-app@latest my-react-native-app --template blank`

- `cd <name>`
- `npm run android`
`npm run ios` (you need to use macOS to build the iOS project - use the Expo app if you need to do iOS development without a Mac)
`npm run web`

9

— Aanmaken project ⁽²⁾

Template types:

- **default:** standaard template, geschikt voor de meeste apps, bevat meerdere schermen, TypeScript geconfigureerd, routing ingesteld.
- **blank:** minimale start-template met weinig extra configuratie, geen navigatie vooraf ingebouwd.
- **blank-typescript:** zelfde als blank, maar met TypeScript.
- **tabs:** template met file-based routing (via de Expo Router) en vooraf ingestelde tabnavigatie; handig voor apps met meerdere schermen.
- **bare-minimum:** template die de native mappen ('android', 'ios') genereert. Je hebt dus toegang tot native code, minder "fully managed" dan de andere.

10

— Migreren van React naar React Native

Wat je kunt **hergebruiken** bij migratie van React naar React Native:

- State management (React hooks).
- API-calls (fetch).
- Utility-functies (helpers, constants).
- Business logica.

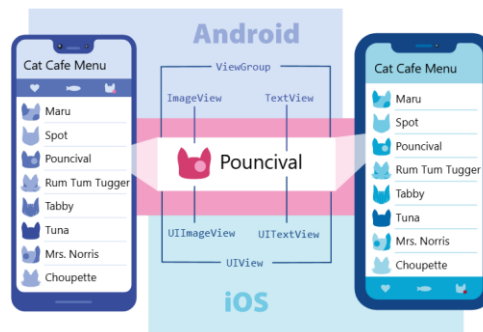
Wat je moet **herschrijven**:

- Alle HTML- en CSS-code herschrijven naar Native componenten en styles.
- Routing met React Navigation.
- Eventuele DOM-manipulatie of web-specifieke code.

11

— GUI-componenten ⁽¹⁾

In Android en iOS is een *view* de basisbouwsteen van de gebruikersinterface: een klein rechthoekig element op het scherm dat kan worden gebruikt om tekst en afbeeldingen weer te geven of te reageren op gebruikersinvoer. Zelfs de kleinste visuele elementen, zoals een regel tekst of een knop, zijn vormen van views. Sommige soorten views kunnen op hun beurt andere views bevatten (geneste views).



12

— GUI-componenten (2)



Enkele basiscomponenten:

Meer info:
<https://reactnative.dev/docs/components-and-apis>

React Native UI Component	Android View	iOS View	Web Analog	Description
<View>	<ViewGroup>	<UIView>	A non-scrolling <div>	A container that supports layout with flexbox, style, some touch handling, and accessibility controls
<Text>	<TextView>	<UITextView>	<p>	Displays, styles, and nests strings of text and even handles touch events
<Image>	<ImageView>	<UIImageView>		Displays different types of images
<ScrollView>	<ScrollView>	<UIScrollView>	<div>	A generic scrolling container that can contain multiple components and views
<TextInput>	<EditText>	<UITextField>	<input type="text">	Allows the user to enter text

13

— Voorbeeld (1)

Counter-voorbeeld uit hoofdstuk 1 werd hier herwerkt naar een React Native app.

index.js:

```
import { registerRootComponent } from 'expo';

import App from './App';

// registerRootComponent calls AppRegistry.registerComponent('main', () => App);
// It also ensures that whether you load the app in Expo Go or in a native build,
// the environment is set up appropriately
registerRootComponent(App);
```

```
my-react-native-app
├── assets
├── .gitignore
├── App.js
├── app.json
├── Counter.js
├── index.js
├── package-lock.json
└── package.json
```

14

— Voorbeeld (2)

Counter.js:

- Het JSX-gedeelte bevat hier niet langer de elementen `<>` en `</>`, het werd nu vervangen door `<Text>`.

```
import { useState, useEffect } from "react";
import { Text } from "react-native";

function Counter() {
  const [count, setCount] = useState(0);

  useEffect(function () {
    const timer = setInterval(function () {
      setCount((count) => count + 1);
    }, 1000);
    return function () {
      clearInterval(timer);
    };
  }, []);

  return (
    <Text>The counter is set to: {count}</Text>
  )
}

export default Counter;
```

15

— Voorbeeld (3)

App.js:

- Het JSX-gedeelte moet hier starten met een root element `<View>`.
- JSX bevat geen HTML meer.
 - HTML-elementen werden vervangen door React Native elementen.
- Event handler types zijn ook anders, hier gebruiken we `onPress` (en niet langer `onClick`) voor de knop.

```
import Counter from "../Counter.js";
import { StatusBar } from "expo-status-bar";
import { View, StyleSheet, Platform, Button, Alert } from "react-native";
import Constants from "expo-constants";

function click() {
  Alert.alert("Hello");
}

export default function App() {
  return (
    <View style={styles.container}>
      <Counter />
      <Button onPress={click} title="Click me"></Button>
      <StatusBar style="auto" />
    </View>
  );
};
```

16

— Voorbeeld (3)

App.js:

- JSX bevat geen CSS meer.
 - CSS werden vervangen door JavaScript-objecten die styling beschrijven. De attributen en namen zijn ook anders dan die bij CSS.

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: "flex-start",
    alignItems: "flex-start",
    backgroundColor: "#f5f5f5",
    marginTop: Platform.OS === "android" ? Constants.statusBarHeight : 50,
  },
});
```

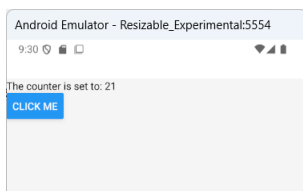
Meer info:

<https://reactnative.dev/docs/style>

17

— Uitvoer

Na ingeven van commando *npm run android*, zie je de uitvoer in de Android Emulator:



18

— Navigatie/routing

De meeste apps hebben meerdere schermen, om van het ene scherm naar het andere te gaan, gebruik je een vorm van routing.

Voor nieuwe projecten:

- `npx create-expo-app@latest --template react-navigation/template`

Voor bestaande projecten:

- `npm install @react-navigation/native @react-navigation/native-stack`

Meer info met codevoorbeeld: <https://reactnative.dev/docs/navigation>

19

— Referenties

- <https://docs.expo.dev/get-started/set-up-your-environment>
- <https://reactnative.dev/docs/getting-started>
 - <https://reactnative.dev/docs/components-and-apis>
 - <https://reactnative.dev/docs/style>
 - <https://reactnative.dev/docs/navigation>

20